Regular Expression

Objective

OBJECT™ TECHNOLOGIES

- What is reular Expression
- Methods
- Special Characters
- Quantifiers
- Special Sequences
- Others
- Examples

Regular Expression

OBJECT™ TECHNOLOGIES

- A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.

- RegEx can be used to check if a string contains the specified search pattern.

- **RegEx Module**
- Python has a built-in package called re, which can be used to work with Regular Expressions.
- import re

Methods

OBJECT™ TECHNOLOGIES

Special characters

.	match any char except newline (eg., ac..ve)
^	match at beginning of string (eg., ^active)
$	match at end of string (eg, state$)
[3a-c]	match any char (ie., 3 or a or b or c)
[^x-z1]	match any char except x, y, z or 1
A|S	match either A or S regex
()	capture & match a group of chars (eg., (8097ba))
\	escape special characters

Quantifiers

*	match 0 or more occurrences (eg., py*)
+	match 1 or more occurrences (eg., py+)
?	match 0 or 1 occurrences (eg., py?)
{m}	match exactly m occurrences (eg., py{3})
{m,n}	match from m to n occurrences (eg., py{1,3})
{,n}	match from 0 to n occurrences (eg., py{,3})
{m,}	match m to infinite occurrences (eg., py{3,})
{m,n}	match m to n occurrences, but as few as possible (eg., py{1,3}?)

Special sequences

\A	match occurrence only at start of string
\Z	match occurrence only at end of string
\b	match empty string at word boundary (e.g., between \w and \W)
\B	match empty string not at word boundary
\d	match a digit
\D	match a non-digit
\s	match any whitespace char: [\t\n\r\f\v]
\S	match any non-whitespace char
\w	match any alphanumeric: [0-9a-zA-Z_]
\W	match any non-alphanumeric
\g<id>	matches a previously captured group
(?:A)	match expression represented by A (non-capture group)
A(?=B)	match expression A only if followed by B
A(?!B)	match expression A only if not followed by B
(?<=B)A	match expression A only if it follows B
(?<!B)A	match expression A only if not preceded by B
(?aiLmsux)	where a, i, L, m, s, u, and x are flags:

a	=	match ASCII only
i	=	make matches ignore case
L	=	make matches locale dependent
m	=	multi-line makes ^ and $ match at the beginning/end of each line, respectively
s	=	makes '.' match any char including newline
u	=	match unicode only
x	=	verbose increases legibility by allowing comments & ignoring most whitespace

- The re.compile() method
- re.compile(pattern, repl, string):

- We can combine a regular expression pattern into pattern objects, which can be used for pattern matching. It also helps to search a pattern again without rewriting it.

Example

```
import re
pattern=re.compile('KIT')
result=pattern.findall('KIT Know-it KIT')
print (result)
result2=pattern.findall('KIT is most popular training institute')
print (result2)

Output
['KIT', 'KIT']
['KIT']
```


www.objectiveo.in

```
import re
def text_match(text):
    patterns = 'ab*' #for 0 or more b
    # 'ab+'  one or more b
    #ab{3} followed by 3 b's
    if re.search(patterns, text):
        return 'Found a match!'
    else:
        return('Not matched!')

print(text_match("c"))
print(text_match("ac"))
print(text_match("abc"))
print(text_match("abbc"))
print(text_match("a"))
print(text_match("ed"))
```


www.object.co.in

#[^...] Matches any single character not in brackets

#^ Matches beginning of line.

```
import re
txt = "The rain in Spain"
#Check if the string has other characters than a, r, or n:
x = re.findall("[^arn]", txt)
#y = re.findall("^[The]", txt)
#print(y)
print(x)
if x:
  print("Yes, there is at least one match!")
else:
  print("No match")
```


www.object.co.in

#sequences of lowercase letters joined with a underscore

```
import re
def text_match(text):
    patterns = '^[a-z]+_[a-z]+$'
    if re.search(patterns, text):
        return 'Found a match!'
    else:
        return('Not matched!')

print(text_match("aab_cbbbc"))
print(text_match("aab_Abbbc"))
print(text_match("Aaab_abbbc"))
```


www.object.co.in

- #contains only upper and lowercase letters, numbers, and underscores.

```
import re
def text_match(text):
    patterns = '^[a-zA-Z0-9_]*$'
    if re.search(patterns, text):
        return 'Found a match!'
    else:
        return('Not matched!')

print(text_match("The quick brown fox jumps over the lazy dog."))
print(text_match("Python_Exercises_1"))
```


www.object.com

#Find words in given string

```
import re
patterns = [ 'is', 'my', 'nation' ]
text = 'India is my Country. I love my India.'
for pattern in patterns:
    print('Searching for "%s" in "%s" ->' % (pattern, text),)
    if re.search(pattern,  text):
        print('Matched!')
    else:
        print('Not Matched!')
```

- Write a Python program to search the numbers (0-9) of length between 1 to 3 in a given string.

```
import re
results = re.findall(r"([0-9]{1,3})", "Exercises number 1, 12, 13 and 345 are important")
print("Number of length 1 to 3")
print(results)
```


www.object.co.in

Summary

- What is reular Expression
- Methods
- Special Characters
- Quantifiers
- Special Sequences
- Others
- Examples

Thank you